

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

REPORT

Laboratory work No. 4

The xv6 Shell from the Outside

Rotaru Anastasia, FAF-242

Course: **Operating Systems**

Checked by: Patricia Reiţman, asist. univ.

Chişinău, 2026

Contents

1 Goal of the work **2**

2 Part 1. Fix sleep for the current xv6 **2**

3 Part 2. bash versus the xv6 shell **3**
3.1 Comparative table 4
3.2 Observations 4

4 Part 3. Who expands *? A program that prints its argv **5**
4.1 Observations 6

5 Part 4. Read the shell **7**
5.1 Observations 8

6 Conclusion **9**

1 Goal of the work

The primary objective of this laboratory assignment is to explore the internal mechanics and command-processing logic of a classic Unix shell by contrasting xv6's stripped-down command interpreter (`user/sh.c`) with the full-featured GNU `bash` environment on Linux. Throughout this assignment, performed within my development environment on `Laptop-Rotaru`, I investigated argument parsing, standard stream redirection, pipeline creation, string evaluation, wildcard expansion, quote handling, and the distinction between internal shell built-ins and external executable binaries.

2 Part 1. Fix sleep for the current xv6

In modern iterations of xv6 (released after August 2025), the traditional `sleep()` system call has been renamed to `pause()`. To adapt the repository inside my `UbuntuLab` environment, I first verified this prototype inside `kernel/user.h`, modified `user/sleep.c` accordingly, and ensured the utility is properly registered under the `UPROGS` section in the `Makefile`.

```
nastea@UbuntuLab:~/xv6-riscv$ grep -n "pause\|sleep" user/user.h
25:int pause(int);
```

The result confirms that the system call is now called `pause`. The updated `user/sleep.c` source file is structured as follows:

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2) {
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1])); // was sleep() before Aug 2025
  exit(0);
}
```

After recompiling the system image via `make qemu`, I performed functional checks directly inside the xv6 shell:

```
$ sleep 20
$ sleep
usage: sleep <ticks>
```

25:int pause(int);

Figure 1: The grep result in user.h

```c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2) {
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1])); // was sleep() before Aug 2025
  exit(0);
}
```

Figure 2: The updated sleep.c

```
$ sleep 20
$ sleep
usage: sleep <ticks>
```

Figure 3: sleep running in xv6, with and without an argument

3 Part 2. bash versus the xv6 shell

I executed a set of ten comparative commands inside both environments: utilizing the `~/shtest` directory containing the `README` file under `bash` on my Linux virtual machine (`UbuntuLab`), and subsequently inside the xv6 QEMU shell.

3.1 Comparative table

Command	Tests	bash (Linux)	xv6 shell
`echo hello > f; cat < f`	redirection	Prints `hello`. Correctly redirects descriptors 1 and 0.	Prints `hello`. Redirects files via `open` and `dup`.
`ls | grep READ`	pipe	Prints `README`. Connects stdout to stdin via `pipe`.	Prints `README`. Complete implementation of anonymous pipes.
`(echo a; echo b) | wc`	grouping, list	Prints `2 2 4` (lines, words, bytes).	Prints `2 2 4`. Evaluates the grouped subshell.
`echo one; echo two`	sequence	Prints `one`, then `two`.	Prints `one`, then `two` (the `;` operator).
`echo bg &`	background	Prints the background PID `[1] 11330` and `bg`.	Prints `bg` asynchronously, without blocking the prompt.
`ls *`	globbing	Expands `*` to the files in the folder (`README f`).	Error: `ls: cannot open *`. Looks literally for a file named `*`.
`echo $HOME`	variables	Replaces the variable with the path `/home/nastea`.	Prints the literal string `$HOME`. No variable support.
`echo "a b"`	quotes	Prints `a b`. Quotes are removed by the shell.	Prints `"a b"` with the quotes included.
`mkdir d; cd d; ls ..`	PATH	Creates `d`, enters it and runs `ls ..` found via `$PATH`.	`ls ..` fails (`exec ls failed`) because there is no `PATH` variable.
`arrow up, Tab, Ctrl+R`	history, completion	Autocompletes names and navigates history.	Prints raw control characters (no advanced terminal support).

3.2 Observations

1. **Core features versus advanced extensions:** Both interpreters handle standard input/output redirection (`<`, `>`), pipelines (`|`), sequential command separation (`;`), and background task execution (`&`) natively via underlying system primitives. Conversely, advanced features like globbing expansion (`*`), environment variable resolution (`$`), quote stripping, automatic binary discovery via `PATH`, command history,

and tab completion are exclusive to `bash`.

2. **The behavior of `ls` following `cd d` in xv6:** Because xv6 lacks a dynamic `PATH` environment variable, the shell cannot automatically locate binaries outside the current working directory. Consequently, executing `ls` after navigating into directory `d` fails unless the explicit relative path is provided, such as `../ls ...`

3. **Practical implications:** The absence of command-line history and auto-completion notably reduces efficiency during interactive sessions. From an implementation perspective, stripping literal quotes during argument parsing represents one of the more straightforward enhancements that could be integrated into `sh.c`.

```
nastea@UbuntuLab:~/shtest$ echo hello > f; cat < f
hello
nastea@UbuntuLab:~/shtest$ ls | grep READ
README
nastea@UbuntuLab:~/shtest$ ls *
README  f
```

Figure 4: The commands run in bash

```
$ echo hello > f; cat < f
hello
$ ls | grep READ
README
$ ls *
ls: cannot open *
$ echo $HOME
$HOME
```

Figure 5: The same commands run in xv6

```
$ mkdir d
$ cd d
$ ls ..
exec ls failed
$ ../ls ..
```

Figure 6: ls failing after cd d, and the working alternative

4 Part 3. Who expands *? A program that prints its argv

To analyze how command-line arguments are transmitted into the `argv[]` vector, I created custom diagnostic programs: `args.c` for xv6 and `args_linux.c` for Linux.

Results in Linux (`bash`):

```
nastea@UbuntuLab:~/shtest$ ./args one     two     three
argc = 4
argv[0] = [./args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]

nastea@UbuntuLab:~/shtest$ ./args *.c "x y"
argc = 3
argv[0] = [./args]
argv[1] = [args_linux.c]
argv[2] = [x y]

nastea@UbuntuLab:~/shtest$ ./args $HOME
argc = 2
argv[0] = [./args]
argv[1] = [/home/nastea]

nastea@UbuntuLab:~/shtest$ ./args 'single quoted'
argc = 2
argv[0] = [./args]
argv[1] = [single quoted]
```

Results in xv6:

```
$ args one     two     three
argc = 4
argv[0] = [args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]

$ args *.c "x y"
argc = 4
argv[0] = [args]
argv[1] = [*.c]
argv[2] = ["x]
argv[3] = [y"]

$ args $HOME
argc = 2
argv[0] = [args]
argv[1] = [$HOME]
```

4.1 Observations

1. **Contrasting the execution of `args *.c "x y"`:**
   - Under `bash`, the command interpreter processes the wildcard expression, expanding `*.c` into matching filenames, and strips quotation marks around `"x y"`, resulting in 3 distinct arguments.
   - In contrast, the xv6 shell passes both arguments raw and uninterpreted, yielding 4 elements since the internal space splits the quoted string.

2. **Responsibility for expansion and parsing:** Filename expansion and quote removal are strictly responsibilities of the shell interpreter rather than the kernel or individual executable binaries. Since xv6's minimalist shell contains no such logic, every token is forwarded verbatim to the process image via the `exec` call.

3. **Whitespace handling:** Both command line interpreters correctly treat consecutive whitespace characters as argument delimiters, filtering out extra spacing to pass clean individual parameters.

```
        $U/_args\
```

Figure 7: The `$U/_args` line added to UPROGS in the Makefile

```
$ args one     two     three
argc = 4
argv[0] = [args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]
```

Figure 8: args output in xv6

```
nastea@UbuntuLab:~/shtest$ ./args one     two     three
argc = 4
argv[0] = [./args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]
```

Figure 9: args*linuxoutputinbash*

5 Part 4. Read the shell

I conducted a thorough code inspection of `user/sh.c` (totaling 499 lines) utilizing standard system utilities like `wc` and `grep`:

```
nastea@UbuntuLab:~/xv6-riscv$ wc -l user/sh.c
499 user/sh.c

nastea@UbuntuLab:~/xv6-riscv$ grep -n "#define" user/sh.c
8:#define EXEC  1
9:#define REDIR 2
10:#define PIPE  3
11:#define LIST  4
12:#define BACK  5
14:#define MAXARGS 10

nastea@UbuntuLab:~/xv6-riscv$ grep -n "case " user/sh.c
```

```
75:  case EXEC:
83:  case REDIR:
93:  case LIST:
101:  case PIPE:
125:  case BACK:
...
nastea@UbuntuLab:~/xv6-riscv$ grep -n "cd" user/sh.c | head
170:        fprintf(2, "cannot cd %s\n", cmd + 3);
```

5.1 Observations

1. **The five core command types and their syntactic triggers:**
   - `EXEC` (1) – simple commands with space-separated tokens.
   - `REDIR` (2) – stream redirection triggered by `<` and `>`.
   - `PIPE` (3) – inter-process communication channels established via `|`.
   - `LIST` (4) – sequential command lists separated by `;`.
   - `BACK` (5) – asynchronous background execution denoted by `&`.
2. **The execution context of `cd`:** The directory change command is handled internally within the main execution loop of `main()` via the `chdir(cmd + 3)` system call. It cannot be implemented as an external binary because spawning a child process via `fork()` would only alter the working directory of the child, leaving the parent shell unaffected.
3. **Absence of pattern matching:** A scan of `sh.c` confirms that no parsing logic exists for symbols like `*` or `$`, fully explaining why xv6 forwards these characters unaltered.

```
8:#define EXEC  1
9:#define REDIR 2
10:#define PIPE  3
11:#define LIST  4
12:#define BACK  5
```

Figure 10: The command kinds defined in sh.c

```
75:  case EXEC:
83:  case REDIR:
93:  case LIST:
101:  case PIPE:
125:  case BACK:
```

Figure 11: Where each kind is run

```
170:        fprintf(2, "cannot cd %s\n", cmd + 3);
```

Figure 12: The code that handles cd

6 Conclusion

Through this laboratory assignment, I gained a comprehensive understanding of Unix command interpreter architecture by contrasting the minimalist xv6 shell with GNU `bash`. The practical experiments highlighted the clear architectural boundary between kernel-level primitives (`fork`, `exec`, `pipe`, `chdir`) and shell-level preprocessing. Tasks such as wildcard expansion, environment variable substitution, and quote removal are entirely handled by the shell prior to program execution. Examining `sh.c` clarified why built-in commands like `cd` must reside directly inside the interpreter core, deepening my grasp of foundational process management in operating systems.